

Especificação do Sistema de Atendimento

1. Configurações e Ambiente

/requirements.txt

- **ação:** criar
- **descrição:** Arquivo de dependências contendo as bibliotecas necessárias para a execução do sistema em ambiente Python.
- **pseudocódigo:**

```
Flask==3.0.0
Flask-SQLAlchemy==3.1.1
python-dotenv==1.0.0
gunicorn==21.2.0
```

/.env.example

- **ação:** criar
- **descrição:** Template para as variáveis de ambiente necessárias para a configuração e provisionamento inicial do sistema.
- **pseudocódigo:**

```
SECRET_KEY=string_aleatoria_segura_aqui
DATABASE_URI=sqlite:///app/db/atendimento.db
DEBUG_MODE=True
PROPRIETARIO_EMAIL=admin@empresa.com
PROPRIETARIO_PASSWORD=senha_segura_inicial
```

/config.py

- **ação:** criar
- **descrição:** Módulo de configuração que carrega as variáveis de ambiente e define as configurações globais da aplicação Flask.
- **pseudocódigo:**

```
IMPORTAR os
IMPORTAR dotenv
EXECUTAR dotenv.load_dotenv()

CLASSE Config:
```

```
DEFINIR SECRET_KEY COMO os.getenv("SECRET_KEY")
DEFINIR SQLALCHEMY_DATABASE_URI COMO os.getenv("DATABASE_URI")
DEFINIR DEBUG COMO BOOLEANO(os.getenv("DEBUG_MODE"))
DEFINIR PROPRIETARIO_EMAIL COMO os.getenv("PROPRIETARIO_EMAIL")
DEFINIR PROPRIETARIO_PASSWORD COMO os.getenv("PROPRIETARIO_PASSWORD")
```

2. Inicialização e Infraestrutura

/app/database.py

- **ação:** criar
- **descrição:** Inicialização da instância do SQLAlchemy responsável pelo mapeamento objeto-relacional (ORM).
- **pseudocódigo:**

```
IMPORTAR SQLAlchemy DE flask_sqlalchemy
INSTANCIAR db = SQLAlchemy()
```

/app/__init__.py

- **ação:** criar
- **descrição:** Função de fábrica (factory) da aplicação Flask. Responsável por inicializar extensões e registrar blueprints (controladores).
- **pseudocódigo:**

```
IMPORTAR Flask
IMPORTAR Config DE config
IMPORTAR db DE app.database

FUNCAO create_app():
    INSTANCIAR app = Flask(__name__)
    CARREGAR_CONFIGURACOES(app, Config)

    INICIALIZAR db COM app

    # Registro de Blueprints (Controllers)
    REGISTRAR_BLUEPRINT(auth_bp)
    REGISTRAR_BLUEPRINT(proprietario_bp)
    REGISTRAR_BLUEPRINT(administrador_bp)
    REGISTRAR_BLUEPRINT(atendente_bp)
    REGISTRAR_BLUEPRINT(cliente_bp)

    RETORNAR app
```

/run.py

- **ação:** criar
- **descrição:** Script principal de entrada da aplicação. Inicializa o servidor e cria o usuário Proprietário inicial caso o banco esteja vazio.
- **pseudocódigo:**

```
IMPORTAR create_app DE app
IMPORTAR db DE app.database
IMPORTAR UsuarioModel DE app.models.usuario_model
IMPORTAR Config DE config

app = create_app()

SE _ARQUIVO_EXECUTADO_DIRETAMENTE:
    COM CONTEXTO_DA_APLICACAO(app):
        db.create_all() # Cria as tabelas se não existirem

        SE PROPRIETARIO_INICIAL_NAO_EXISTE:
            novo_proprietario = CRIAR UsuarioModel(
                email=Config.PROPRIETARIO_EMAIL,
                nome="Proprietário Principal",
                role="PROPRIETARIO"
            )
            novo_proprietario.set_senha(Config.PROPRIETARIO_PASSWORD)
            SALVAR_NO_BANCO(novo_proprietario)

        INICIAR_SERVIDOR(app, host="0.0.0.0", port=5000)
```

/Dockerfile

- **ação:** criar
- **descrição:** Definição da imagem do container para empacotamento da aplicação, garantindo portabilidade.
- **pseudocódigo:**

```
DEFINIR_IMAGEM_BASE python:3.10-slim
DEFINIR_DIRETORIO_TRABALHO /app
COPIAR requirements.txt PARA .
EXECUTAR pip install --no-cache-dir -r requirements.txt
COPIAR todo_conteudo PARA .
EXPOR_PORTA 5000
DEFINIR_COMANDO_INICIAL ["python", "run.py"]
```

3. Camada de Dados (Models)

/app/models/usuario_model.py

- **ação:** criar
- **descrição:** Definição da entidade de usuários unificada, utilizando um campo 'role' para determinar os privilégios.
- **pseudocódigo:**

```
IMPORTAR db DE app.database

CLASSE UsuarioModel(db.Model):
    DEFINIR_NOME_TABELA "usuarios"

    COLUNA id COMO INTEIRO, CHAVE_PRIMARIA
    COLUNA nome COMO TEXTO, NAO_NULO
    COLUNA email COMO TEXTO, UNICO, NAO_NULO
    COLUNA senha_hash COMO TEXTO, NAO_NULO
    COLUNA role COMO ENUM("PROPRIETARIO", "ADMINISTRADOR", "ATENDENTE", "CLIENTE"), NAO_NULO
    COLUNA criado_por_id COMO INTEIRO, CHAVE_ESTRANGEIRA("usuarios.id"), NULO_PERMITIR_NULO

    METODO set_senha(senha_plana):
        ATRIBUIR self.senha_hash = GERAR_HASH_CRIPTOGRAFICO(senha_plana)

    METODO check_senha(senha_plana):
        RETORNAR COMPARAR_HASH(self.senha_hash, senha_plana)
```

/app/models/ticket_model.py

- **ação:** criar
- **descrição:** Definição da entidade de solicitações (tickets) criadas pelos clientes e respondidas pelos atendentes.
- **pseudocódigo:**

```
IMPORTAR db DE app.database
IMPORTAR datetime

CLASSE TicketModel(db.Model):
    DEFINIR_NOME_TABELA "tickets"

    COLUNA id COMO INTEIRO, CHAVE_PRIMARIA
    COLUNA cliente_id COMO INTEIRO, CHAVE_ESTRANGEIRA("usuarios.id"), NAO_NULO
    COLUNA assunto COMO TEXTO, NAO_NULO
    COLUNA descricao COMO TEXTO, NAO_NULO
    COLUNA status COMO ENUM("ABERTO", "EM_ATENDIMENTO", "RESOLVIDO"), PADRAO="ABERTO"
    COLUNA data_criacao COMO DATATIME, PADRAO=datetime.utcnow()
```

```
COLUMNA data_criacao COMO DATAHORA, PADRAO=datetime.utcnow
COLUMNA data_atualizacao COMO DATAHORA, PADRAO=datetime.utcnow, AO_ATUALIZAR=datet:
```

/app/models/log_model.py

- **ação:** criar
- **descrição:** Definição da entidade para armazenamento de logs de operações (auditoria de cadastros e respostas).
- **pseudocódigo:**

```
IMPORTAR db DE app.database
IMPORTAR datetime

CLASSE LogOperacaoModel(db.Model):
    DEFINIR_NOME_TABELA "logs_operacoes"

    COLUMNA id COMO INTEIRO, CHAVE_PRIMARIA
    COLUMNA usuario_id COMO INTEIRO, CHAVE_ESTRANGEIRA("usuarios.id"), NAO_NULO # Quem
    COLUMNA acao COMO TEXTO, NAO_NULO # Ex: "CRIAR_ATENDENTE", "RESPONDER_TICKET"
    COLUMNA tabela_afetada COMO TEXTO, NAO_NULO
    COLUMNA registro_afetado_id COMO INTEIRO, NAO_NULO
    COLUMNA data_hora COMO DATAHORA, PADRAO=datetime.utcnow
```

4. Camada de Controle (Controllers)

/app/controllers/auth_controller.py

- **ação:** criar
- **descrição:** Define as rotas (Blueprints) para login, logout e autocadastro de clientes.
- **pseudocódigo:**

```
IMPORTAR Blueprint, request, session DE flask
IMPORTAR UsuarioModel

auth_bp = Blueprint('auth')

ROTA '/login' METODOS=['POST']:
    email = request.form['email']
    senha = request.form['senha']

    usuario = BUSCAR UsuarioModel POR email
    SE usuario EXISTE E usuario.check_senha(senha):
        session['user_id'] = usuario.id
        session['user_role'] = usuario.role
```

```

        session['user_role'] = usuario.role
        RETORNAR REDIRECIONAR_PARA_PAINEL(usuario.role)
    SENA0:
        RETORNAR ERRO "Credenciais Inválidas"

ROTA '/cadastro-cliente' METODOS=['POST']:
    DADOS = request.form
    SE BUSCAR UsuarioModel POR DADOS.email:
        RETORNAR ERRO "Email já em uso"

    novo_cliente = CRIAR UsuarioModel(nome=DADOS.nome, email=DADOS.email, role="CLIENTE")
    novo_cliente.set_senha(DADOS.senha)
    SALVAR_NO_BANCO(novo_cliente)

    RETORNAR SUCESSO "Cliente cadastrado"

```

/app/controllers/administrador_controller.py

- **ação:** criar
- **descrição:** Blueprint com lógicas de CRUD de atendentes e geração de relatórios. Inclui verificação de permissão (RBAC).
- **pseudocódigo:**

```

IMPORTAR Blueprint, request, session DE flask
IMPORTAR UsuarioModel, TicketModel, LogOperacaoModel

admin_bp = Blueprint('admin')

DECORADOR verificar_permissao_admin():
    SE session['user_role'] != 'ADMINISTRADOR': ABORTAR(403)

ROTA '/admin/atendentes/novo' METODOS=['POST']:
    verificar_permissao_admin()
    DADOS = request.form

    novo_atendente = CRIAR UsuarioModel(nome=DADOS.nome, email=DADOS.email, role="ATENDENTE")
    novo_atendente.set_senha(DADOS.senha)
    SALVAR_NO_BANCO(novo_atendente)

    log = CRIAR LogOperacaoModel(usuario_id=session['user_id'], acao="CRIAR_ATENDENTE")
    SALVAR_NO_BANCO(log)

    RETORNAR SUCESSO "Atendente criado"

```

5. Camada de Apresentação (Views)

/app/templates/admin/criar_atendente.html

- **ação:** criar
- **descrição:** Template base utilizando Jinja2 e importando os assets do Bootstrap para consistência visual.
- **pseudocódigo:**

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
    <meta charset="UTF-8">
    <title>{% block title %}Sistema de Atendimento{% endblock %}</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body>
    <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
        </nav>

    <div class="container mt-4">
        {% block content %}
        {% endblock %}
    </div>

    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js">
</body>
</html>
```